

An Algorithm to Compute the All-terminal Reliability Measure

Héctor Cancela*,

*Facultad de Ingeniería, Universidad de la República,
Montevideo, Uruguay*

Gerardo Rubino and María E. Urquhart

Projet ARMOR, IRISA, Campus de Beaulieu, Rennes , France

Abstract

In the evaluation of the reliability of a multi-component system, the most important among several models fit into the class of stochastic graph models, for which an exact evaluation of reliability metrics is costly since these are classed in the NP-hard family. As a consequence, many computation methods exist. This paper considers the all-terminal reliability measure R , which is extremely useful in the context of communication networks. It represents the probability that network nodes can exchange messages, taking into account possible failures of network components (nodes and links). The algorithm has the property of working with a bounded space, in fact, with space complexity linear in the size of the problem, an improvement over existing techniques which are exponential in space.

Key words

Reliability evaluation, Stochastic graphs, Telecommunications.

1. Introduction

Reliability models evaluate the capacity of a given system or product to operate correctly (according to specifications). In particular, when the system is made up of a number of individual components, it is necessary to consider their interactions and how possible failures may affect the operation of the whole system.

Stochastic graphs are an important class of models, which can represent a wide variety of system structures and operational criteria. The system is represented as a network G whose nodes are perfect and whose links fail randomly and independently (both links and nodes can be subject to failure). Assume that $G=(V,E)$ is an undirected graph, connected and without loops; the vertex set V corresponds to nodes of the network, and the edge set E corresponds to links. When the link states (operational or failed) are random, the success of communication between any pair of nodes is also a random event.

The probability R of this event is called the all-terminal reliability of the network, and equivalently the parameter $Q=1-R$ is called the all-terminal unreliability. The evaluation of this problem has received considerable attention from the research community (see [9] and [12] for many references). One of the reasons is that in the general case, this problem is NP-hard [3,4,11].

Among the many published techniques to analyse these metrics, the method proposed in [1] the source-terminal reliability performs well and [2] presents some improvements of the basic technique. In [10] an implementation method is suggested that requires a small amount of memory. This paper presents a similar algorithm for the all-terminal reliability measure R .

This paper is organised as follows. Notation and definitions are introduced in Section 2. In Section 3 the algorithm is described for the evaluation of R . Section 4 presents computational results and the last section is devoted to conclusions.

2. Global notation and definitions

The following notation will be used in the formulation of the problem:

- $G=(V,E)$: an undirected network, where $V=\{1,...,n\}$ is the node-set and $E=\{e_1,...,e_m\}$ is the link-set of the network;
- X_e : the binary random variable "state of link e in G ", defined by $X_e=1$ if link e is up (operational), and $X_e=0$ if link e is down (failed);
- r_e : the elementary reliability of link e , that is, $r_e = P(X_e=1)$;
- q_e : the unreliability of link e , $q_e=1 - r_e$;
- $X=(X_1,...,X_m)$: the random network state vector also called random network configuration [12];
- Φ : the structure function of the model, that is, the function of $\{0,1\}^m$ in $\{0,1\}$ valued by 1 if the network deduced from G by deleting down links is connected, 0 otherwise;
- $\Phi(X)$ the binary random variable "state of network G ", that is; $\Phi(X)=1$ if the graph obtained from G by removing each failed link in X is connected, and $\Phi(x) = 0$ otherwise;

- $R=P(\Phi(X)=1)$ the *all-terminal reliability* parameter of network G ;
- A tree is a minimal set of components (links) which connect all nodes in the network. A tree has no cycles.

3. Description of the algorithm

The basic method comes from [1] and the general description is presented in [10], where an improved version of the initial algorithm is also given, which does not require the generation of an intermediate list of events. In this paper, the idea is to compute the reliability R for the connection between network nodes under the assumption of independent behaviour by the different links.

Let C denote the event "the (random) graph is connected"; then it follows that $R=P(C)$. If $\{\pi_1, \dots, \pi_t\}$ is the set of trees and P_k denotes the event "every link in the k^{th} tree is working correctly", then $C=\bigcup_{1 \leq k \leq t} P_k$.

Observe that the probability of events P_k can be immediately generated from the independence hypothesis but since in general events $\{P_1, P_2, \dots\}$ are not disjoint, the formula above is not very useful to compute the reliability R .

Considerable work in the family of direct approaches to reliability computation has been inspired by the idea of constructing a partition of the event C , that is, finding a family of events $\{B_k\}$ such that: $C = \bigcup_k B_k$ with $B_i \cap B_j = \emptyset, \forall i \neq j$. With such a partition, R can be calculated using the following equation: $R=P(C)=\sum_k P(B_k)$.

The method proposed here is based on the same idea. It has the property of constructing the partition $\{B_k\}$ simultaneously with the exploration of the graph without calculating the list $\{P_1, P_2, \dots\}$ thus requiring a small amount of memory to work. In particular, the total amount of space can be statistically calculated from the degrees of the nodes and then allocated.

A partition of C is constructed in terms of events called branches [1]. The branches will be denoted by sequences of symbols taken from the alphabet $V \cup G \cup NV$ with $NV=\{-x / x \in V\}$ and $GV=\{gx / x \in V\}$. For instance, if $V=\{1,2,3\}$ then $NV = \{-1, -2, -3\}$ and $GV = \{g1, g2, g3\}$.

The algorithm consists of moving continuously along a single branch, which is transformed from time to time into an element of the partition; such an element will be called a finished branch. At this point, the probability of this event is computed and cumulated, and the process continues. When the algorithm ends, the set of all the finished branches generated is a partition of C and the cumulated probability is R .

Any branch, finished or not, is represented by a sequence $b=(b_1, b_2, \dots, b_h)$ with the following meaning. Assume that $x, y, \dots$ are the points of V . The elements of NV are represented by $-x, -y, \dots$ and those of GV by $gx, gy, \dots$ where $x, y, \dots \in V$. The first element is $b_1=s$, where s is chosen arbitrarily. The consecutive sequence (x, y)

of $V \times V$ means that x and y are adjacent nodes and that the event b implies the event "link (x,y) is working". The consecutive sequence (gx,y) has exactly the same meaning (we will see later the use of the prefix g). A sequence of consecutive symbols of the form $(u, -y_1, -y_2, \dots, -y_L)$ where $u = x$ or $u = gx$ means that x is adjacent to $y_1, y_2, \dots, y_L$ and that link (x,y_l) does not work, for $l=1,2,\dots,L$. If the subsequence is $(u, -y_1, -y_2, \dots, -y_L, v)$ where u means either x or gx then the meaning is as before with, in addition, $v=z \in V$ or $v=gz \in GV$ and in the first case, x and z are adjacent and link (x,z) works.

A finished branch is a branch in which all nodes in the graph are present. As a consequence of the method, in every branch $b=(b_1, b_2, \dots, b_H)$, if $H > 1$ then $b_H \neq s$ (but the case $b_H = gs$ is possible). Given a finished branch b , its probability is given by:

$$P(b) = \prod_{e_i \in \text{working}(b)} r_{e_i} \cdot \prod_{e_j \in \text{failed}(b)} (1-r_{e_j})$$

where $\text{working}(b)$ and $\text{failed}(b)$ are the sets of links defined respectively as working or down in b according to the rules defining the meaning of the representation.

The algorithm starts with the initial branch $b=(s)$, where s is an arbitrarily chosen node. The main loop consists of looking at the last symbol of b and carrying out some actions depending on three possible cases. The algorithm is described in detail below. For any branch b , the following notation will be used:

- $\forall h: b_h \in NV, V(b_h)=x$ if $b_h=x, V(b_h) = y$ if $b_h = gy$.
- $\forall h < H: \text{next}(h)=\min\{i / h < i \leq H, b_i \in NV\}$
- $\forall h > 1: \text{previous}(h)=\max \{i / 1 \leq i < h, b_i \in NV\}$
- b_L =the last reached node in branch b ; $L=H$ if $b_H \in V \cup GV$, $L=\text{previous}(H)$ if $b_H \in NV$.
- $\forall h: b_h \in NV$, let $x = V(b_h)$; then:
 - $E(h) = \{y \text{ adjacent to } x / \text{there is no } k < h \text{ such that } b_k=y \text{ or } b_k = gy, \text{ there is no } k > h \text{ such that } k < \text{next}(h) \text{ and } b_k = -y\}$.
 - $\forall x \in V, \text{first}(x)=\min\{i/1 \leq i \leq H, b_i = x \text{ or } b_i = gx\}$.
 - $\text{nbNodes}(b)=|\{i \leq H / b_i \in V\}|$.

The pseudo-code for the main loop of the algorithm can be described as follows:

```

b=(s);
H:=1; L:=1; R:=0.0;
Partition_Completed:=false;
repeat
  do cases
    case (nbNodes(b)=|V|)
      R:=R + P(b);
      b:=(b1,...,bH-1, -bH);
    case (bH ∈ NV) and (# of bH in b)=degree(bH)
      Backtrack; /* b can't give rise to a finished branch */
    otherwise
      do cases
        case E(L)={y,...}≠∅ → b:=(b, y);
          H:=H + 1;
          L:=H;
        case E(L)=∅ → i:=previous(first(V(bL)));
          do cases
            case i ≠ 0 → z:=V(bi); b:=(b, gz);
              H:=H + 1;
              L:=H;
            case i=0 → Backtrack;
          endcases
        endcases
      endcases
  until Partition_Completed;
return R;

```

The subroutine Backtrack is used in the algorithm as shown below and used to start the search for a new branch when either there is a finished branch or there is no possibility to finish the branch under construction.

Backtrack subroutine

$j := \max \{i/1 \leq i < L, b_i \in NV, E(i) \neq \emptyset\};$

do cases

case $j=0 \rightarrow \text{Partition_Completed} := \text{true};$

case $j \neq 0 \rightarrow k := \text{next}(j);$

$y := V(b_k);$

$b := (b_1, \dots, b_{k-1}, -y)$

$H := k;$

$L := \text{previous}(H);$

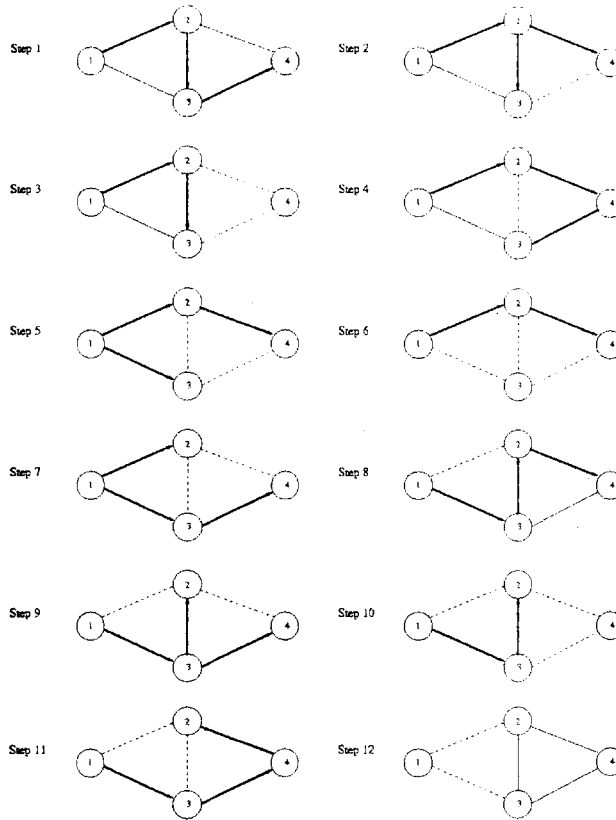
endcases

Instead of looking at the evolution of the single current branch b , the algorithm constructs a tree T with node set $V \cup NV \cup GV$ and root s , as in [1]. There will always be a current branch of T in construction, identical to the current b . When b grows by the addition of a symbol, the corresponding branch of T does the same. When a "backtrack" happens, a branch $b = (b_1, \dots, b_{k-1}, y, \dots)$ is transformed into $(b_1, \dots, b_{k-1}, -y)$; in T , the current branch gives birth to a new one, having the first $k-1$ nodes in common and ending in $-y$.

The algorithm backtracks when communication between the nodes of G cannot happen in the current branch b . It looks for a node x with a non empty E set to continue the process, that is, to build a new branch in T . If such a node x exists, it is represented in b by the symbol gx (g as in growing). When this is no longer possible, the algorithm ends. If a new branch of the tree is built from a node x in position h of b , the new node y introduced in the tree is chosen from the elements of $E(h)$.

Figure 1 shows a trace of finished branches and backtracking points of the algorithm, when evaluating the all-terminal reliability of a simple network, usually known as the "bridge" (the topology of the bridge network can be seen in the same figure); we take as starting point for the algorithm the node labelled 1.

In order to give a more intuitive understanding of the text notation for the branches, the graphical representation is also provided. For each branch there is an associated graph, where operational links are shown as thick lines, failed links as dashed lines, and links represented as thin continuous lines are not part of the current branch. In this case, the evaluation completes after generating eight finished branches.



	Constructed branch	Operation
Step 1	(1, 2, 3, 4)	$R = r_{12} r_{23} r_{34}$
Step 2	(1, 2, 3, 4, g2, 4)	$R = R + r_{12} r_{23} q_{34} r_{24}$
Step 3	(1, 2, 3, 4, g2, 4)	Backtrack
Step 4	(1, 2, 3, g2, 4, 3)	$R = R + r_{12} q_{23} r_{24} r_{43}$
Step 5	(1, 2, 3, g2, 4, 3, g1, 3)	$R = R + r_{12} q_{23} r_{24} q_{43} r_{13}$
Step 6	(1, 2, 3, g2, 4, 3, g1, 3)	Backtrack
Step 7	(1, 2, 3, 4, g1, 3, 4)	$R = R + r_{12} q_{23} q_{43} r_{13} r_{34}$
Step 8	(1, 2, 3, 2, 4)	$R = R + q_{12} r_{13} r_{32} r_{24}$
Step 9	(1, 2, 3, 2, 4, g3, 4)	$R = R + q_{12} r_{13} r_{32} q_{24} r_{34}$
Step 10	(1, 2, 3, 2, 4, g3, 4)	Backtrack
Step 11	(1, 2, 3, 2, 4, 2)	$R = R + q_{12} r_{13} q_{32} r_{34} r_{42}$
Step 12	(1, 2, 3)	Partition completed

Figure 1: Textual and graphical representation of branches built by the evaluation method

4. Numerical results

The algorithm discussed in the previous section was implemented as a basic functionality of the HEIDI tool. HEIDI is a prototype of a communication network reliability analysis and design tool [5,7]. The tool includes reliability evaluation by exact and Monte Carlo methods, driven by a graphical interface. The user can also use this tool to compute different parameters of the topology, and to study the possibility of improving the network, by a simulated annealing search of alternative configurations. The interface of the tools can be seen in Figure 2.

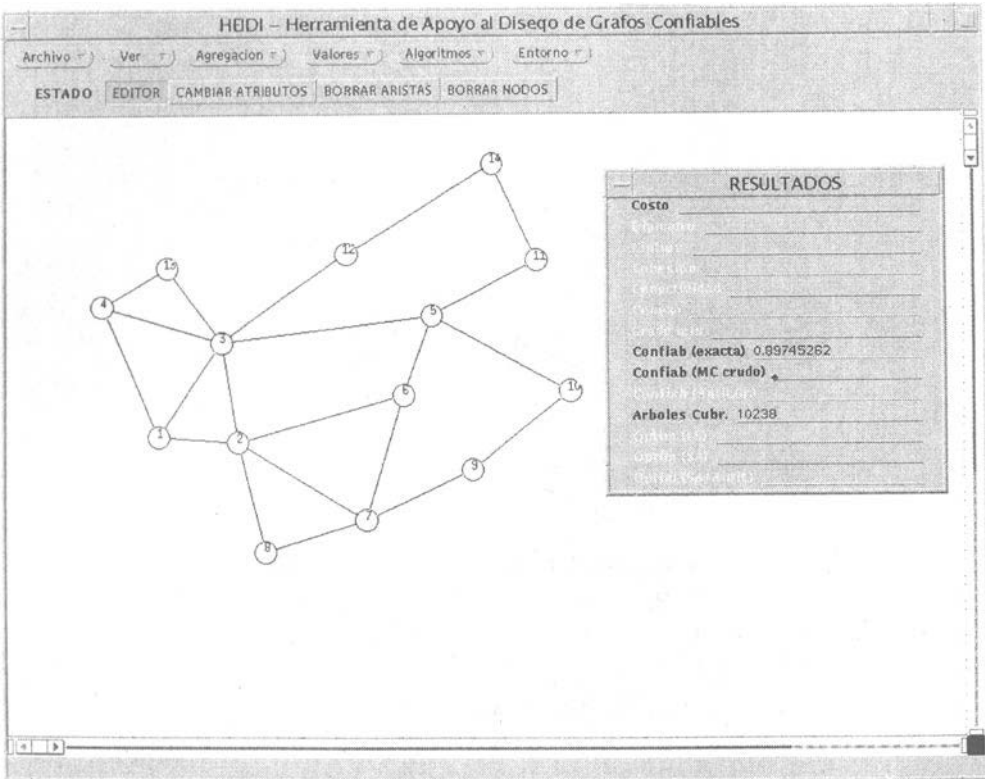


Figure 2: HEIDI tool graphical interface, showing Montevideo's telephonic network core

This research was partly conceived to evaluate the characteristics the optical fiber telephonic network in the city of Montevideo. The HEIDI main window in Figure2, shows the topology of the network core in 1992, when it was manually drawn by the national telecommunications company[6]. In Table 1 the results of numerical tests conducted over this topology are summarised. Links are assumed to be equally reliable (i.e., $r_e=r$ for all $e \in E$), and network reliability is calculated

for the following values of r : 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 0.99, 0.999. In addition, for each value of r the number of branches generated by the algorithm, and computing time (measured in CPU seconds in a Sun Ultra 30, Model 295 are reported.

Table 1: Results for Montevideo's telephonic network.

r	R	Branches	CPU
0.10	5.47E-10	10238	0.78
0.15	7.63E-08	10238	0.78
0.20	2.28E-06	10238	0.79
0.25	2.89E-05	10238	0.81
0.30	2.21E-04	10238	0.80
0.35	0.001073	10238	0.78
0.40	0.004059	10238	0.79
0.45	0.012292	10238	0.77
0.50	0.031082	10238	0.80
0.55	0.067618	10238	0.77
0.60	0.129359	10238	0.80
0.65	0.221286	10238	0.76
0.70	0.342964	10238	0.77
0.75	0.486838	10238	0.77
0.80	0.638861	10238	0.80
0.85	0.781444	10238	0.79
0.90	0.897453	10238	0.81
0.95	0.973323	10238	0.79
0.99	0.998906	10238	0.76
0.999	0.999989	10238	0.77

Network reliability as a function of link reliability r is presented graphically in Figure 3. The algorithm logic is independent from links reliabilities; as the number of branches generated is the same for all values of r , and execution times are also very similar.

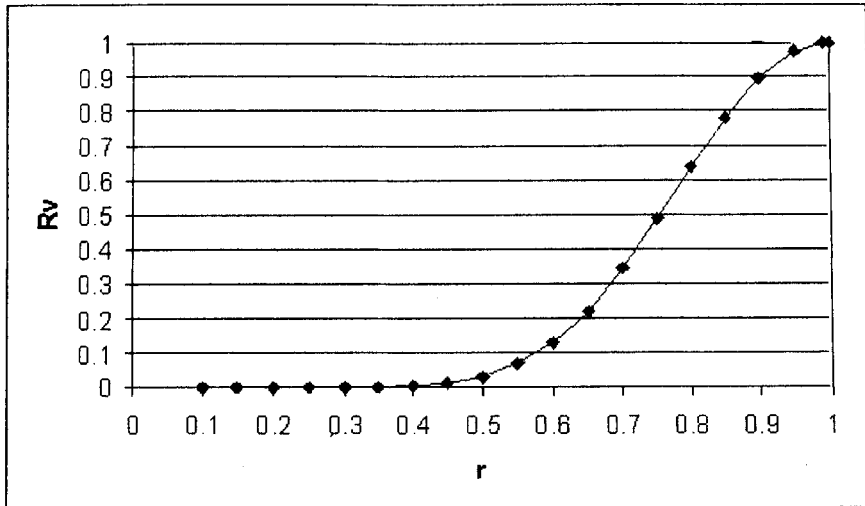


Figure 3: Network reliability as a function of link reliability r (Montevideo's network).

The number of branches is the number of different spanning trees for the network (shown in Figure 2). In fact, a branch can never contain a cycle as the only links that can be added are those having an endpoint not already in the branch; and is a connected subgraph as only links incident to an already present node can be added. As a finished branch must contain all nodes in V , it must contain a spanning tree, plus a number of non-operational links. This is a useful property, as it implies that the arbitrary election of a terminal as a starting point for the algorithm does not influence computational performance.

The algorithm was also used to evaluate a number of important topologies previously used in related work such as the dodecahedron network, shown in Figure 4; a version of the Arpanet, which appears in Figure 5; an undirected version of the double loop network $DL(10,1,3)$, discussed in [8] and shown in Figure 6, and finally K_{10} , the complete graph with 10 nodes.

Table 2 presents results obtained in a Sun Ultra 30 workstation. The first columns identify topologies, showing also the node set and edge set sizes. For the four topologies all edges are assumed to be equally reliable, with common reliability $r_e=0.99$ for the first two cases, $r_e=0.45$ for the third one (the complete graph K_{10}), and $r_e=0.9$ for the last one. The efficiency of the method is independent of this choice; but for values of r_e too close to 1, there may be numerical problems, related to the machine word size. Computed reliability R , and the number of branches built, which is identical to the number of different spanning trees of each network are also presented. Finally, CPU time in seconds is reported. Clearly, the efficiency of the method is strongly dependent upon network size; in particular the complete graph

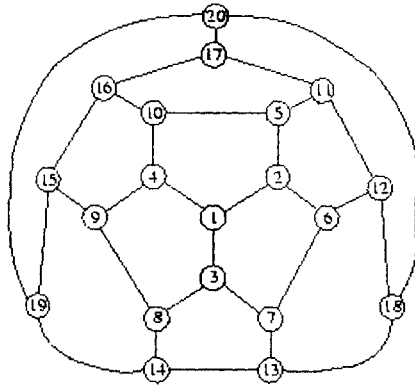


Figure 4: The Dodecahedron network topology

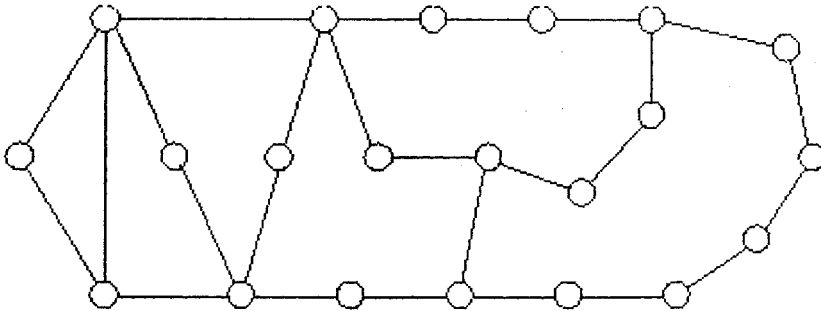


Figure 5: A version of the Arpanet topology (1972)

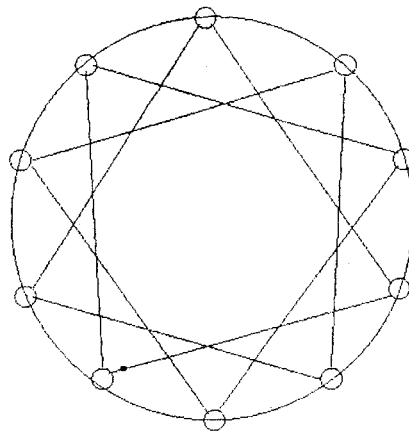


Figure 6: Undirected double loop network topology UDL(10,1,3)

K10, even if it has less nodes than other topologies, but it has more links and in general a more connected topology, which leads to a greater number of branches.

Finding more formal justifications for these behaviour in terms of network structure or of algorithm design is an important issue for future research with a significant potential impact on the method's efficiency.

Table 2: Results of four test topologies.

Topology	IVI	IEI	re	R	Nb. branches	time (sec.)
UDL(10,1,3)	10	20	0.90	0.959898	40500	2
Arpanet	21	26	0.99	0.9973585	15667	7
Dodecahedron	20	30	0.99	0.9999797	5184000	636
K10	10	45	0.45	0.9541519	100000000	3009

5. Conclusions

In this work, a new algorithm has been developed to compute the all-terminal network reliability measure. The method works by implicitly building a partition of the event "the network is operational" in terms of events called branches. As the algorithm moves continuously along a single branch, which is transformed at each step, previously finished branches need not be recorded and therefore the algorithm requires small memory, being linear in the size of the network.

The algorithm starts from any arbitrary node; and the number of generated branches and the total execution time does not depend on this choice.

The method was applied to the evaluation of different graph topologies corresponding to real life communication networks, and well known benchmarks from the literature.

A possible extension of the algorithm to other reliability measures, as well as more test results are particularly interesting themes for future research.

6. Acknowledgement

This work was funded by the INRIA as part of the PAIR project, a partnership effort by the ARMOR project (IRISA - INRIA - France) and the Departamento de Investigación Operativa (FI - UDELAR - Uruguay).

7. References

- [1] S. Hasanuddin Ahmad. A simple technique for computing network reliability. IEEE Tr. Reliability, R-31, 1, April (1982).
- [2] S. Hasanuddin Ahmad and A. T. M. Jamil. A modified technique for computing network reliability. IEEE Tr. Reliability, R-36, 5, December (1987).

- [3] M. O. Ball. The complexity of network reliability computations. *Networks*, 10 (1980). 153-165.
- [4] M. O. Ball. Computational complexity of network reliability analysis: An overview. *IEEE Trans. Reliab.*, R-35, 3, (1986).
- [5] H. Cancela, L. Petingi, G. Rubino, and M. E. Urquhart. HEIDI: a tool for network evaluation and design (text in Spanish). In VIII CLAIO (Operations Research Latin-Iberian-American Congress), Rio de Janeiro, Brazil, ALIO - SOBRAPO, August (1996), 581-586
- [6] H. Cancela, G. Rubino, and M. E. Urquhart. Optimization in communication network design (text in Spanish). In Proceedings of the XIII CILAMCE (Iberian-Latin-American Congress of Computational Methods in Engineering), Porto Alegre, Brasil, November (1992).
- [7] H. Cancela, G. Rubino, and M. E. Urquhart. Evaluation and design of communication networks. In Proceedings of the ICIL'95 (International Congress on Industrial Logistics), Ouro Preto, Brazil,. University of Southampton, UK, and Naval Monterrey School, USA, December (1995).
- [8] F. K. Hwang and P. E. Wright. Survival reliability of some double-loop networks and chordal rings. *IEEE Tr. Computers*, 44, 12, December (1995).
- [9] M. O. Locks and A. Satyarayana editors. Network reliability - the state of the art. *IEEE Trans. Reliab.*, R-35(3), (1986).
- [10] R. Marie and G. Rubino. Direct approaches to the 2-terminal reliability problem. In E. Orhun E. Gelembe and E. Baar, editors, The Third International Symposium on Computer and Information Sciences, , eme, Izmir, Turkey, Ege University, (1988), 740-747.
- [11] J. S. Provan. The complexity of reliability computations in planar and acyclic graphs. *SIAM J. Comput.*, 15(3), Aug. (1986).
- [12] G. Rubino. Network reliability evaluation. In K. Bagchi and Walrand, editors, State-of-the art in performance modeling and simulation, chapter 11. Gordon and Breach Books, (1998).